INSY7315 · TASK 2 · IMPLEMENTATION

SizaniConnect

Food relief operations for Sizani Community Food Network NPC: registration, distribution days, volunteer shifts, stock and donors in one system.

Group 13 · Byron Bousfield (ST10445541)

Client: Sizani Community Food Network NPC (case-study client) · 14 October 2026

sizaniconnect-bb7315.azurewebsites.net

github.com/ByronBousfield1/sizaniconnect

THE PROBLEM

Four problems the paper process causes

01

Households can collect twice

Each site keeps its own paper register, so nothing stops one household collecting at two sites in the same cycle.

02

Nobody can see the stock

Donations are recorded on delivery notes at the depot. Nobody knows what is on hand or about to expire.

03

Shifts run on WhatsApp

Coordinators spend hours confirming volunteers by message, and no hours are recorded for funders.

04

Donor reports take weeks

Every impact report means matching paper registers against delivery notes by hand.

THE SOLUTION

One system for the whole operation

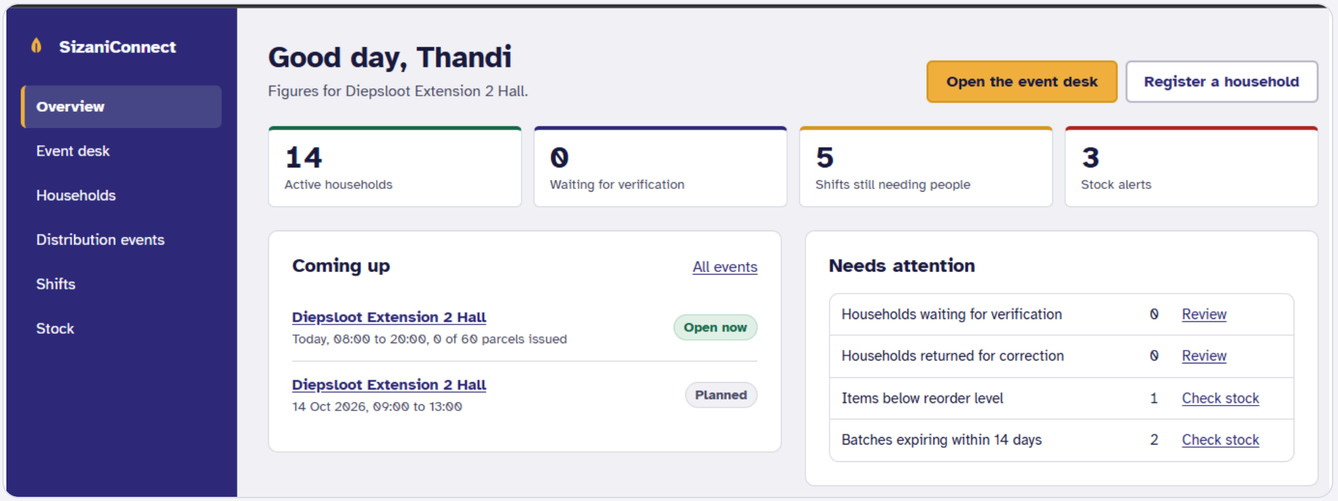
Site coordinator
Registers households and runs the event desk

Volunteer
Signs up for shifts, checks in, records parcels

Operations manager
Verifies households, plans events, stock and reports

Donor
Sees which households their goods reached

Administrator
Accounts, sites and the audit log



The live production site on Azure, signed in as a site coordinator.

20
user stories

22
screens

5
user roles

KEY FEATURES · LIVE DEMO

Live demo: a distribution day

1

Find a household by its slip code, typed carelessly in lower case

2

Record the parcel and see the stock taken, oldest expiry first

3

Scan the same code again: refused, showing who recorded it and when

4

Register someone already registered at another site: blocked

5

Write off expired stock, then run last month's report

6

Open the audit log: every step above is recorded

Logins: coordinator@, volunteer@, ops@, donor@, admin@sizani.test · password Sizani-Demo-2026

KEY FEATURES · FRONT END

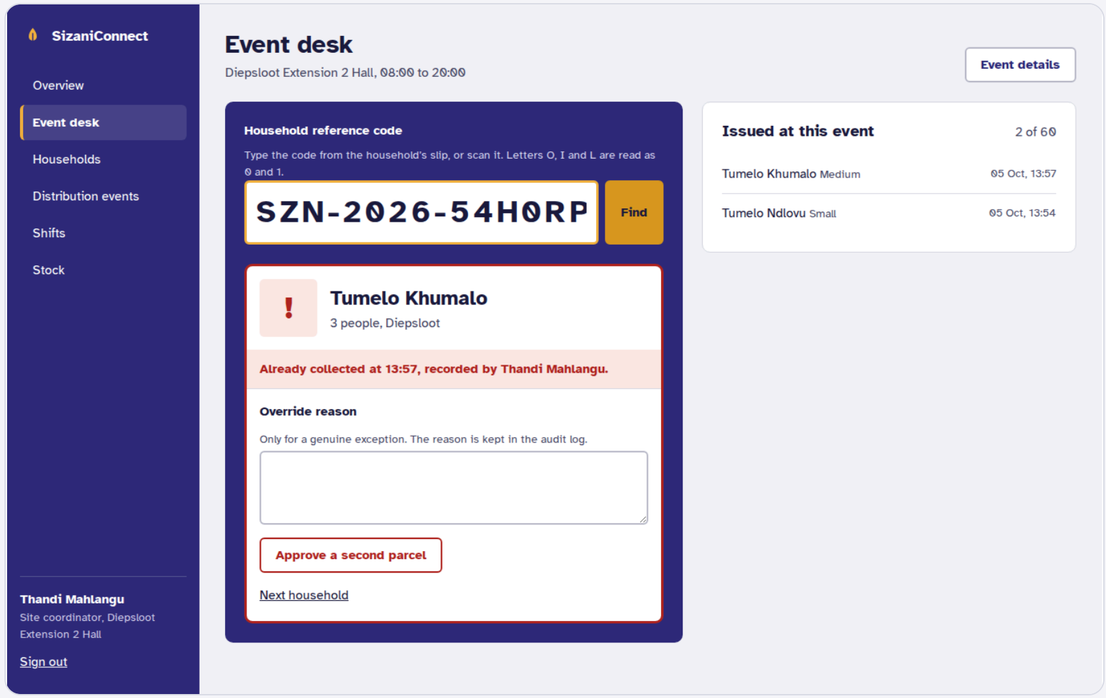
A desk screen built for a phone

22 screens in React 19 and TypeScript, one per screen in the Task 1 journey map.

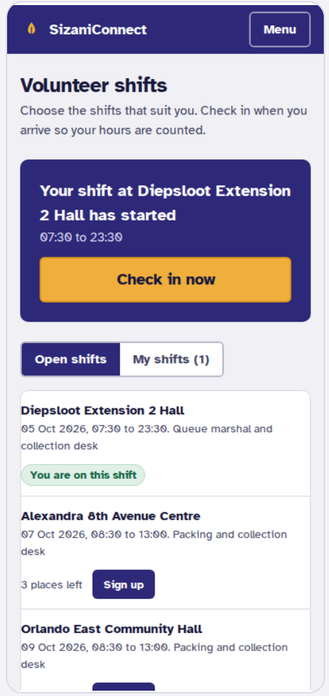
One input, one decision: green means record the parcel, red means stop.

Checked at 360 px wide: no screen scrolls sideways.

Readable outdoors: every text colour 5.5:1 or better, 44 px touch targets, labelled controls.



Event desk: a second parcel is refused



Shifts on a phone

KEY FEATURES · BACK END AND APIS

A layered back end behind a REST API

Browser	React SPA; access token held in memory
HTTP layer	Routes, Zod validation, role policies
Services	Business rules, one transaction each
Domain	Lifecycles, parcel sizes, FEFO stock
Repositories	Parameterised SQL only
PostgreSQL 16	Unique indexes, append-only audit log

Each layer depends only on the one below it.

REST API under /api/v1

Over 50 endpoints. Every error has one shape, a code and a message that says what to do next.

```
POST   /households
GET    /households/by-ref/:code
POST   /collections
POST   /events/:id/open
GET    /reports/monthly.csv
GET    /me/impact
GET    /health
```

KEY FEATURES · DESIGN

Design patterns from Task 1, in code

Pattern	Problem it solves here	Where in the code
Repository	Services read and save data without writing SQL	api/src/repositories/
Unit of Work	Parcel, stock and audit entry are saved together	repositories/index.ts
Strategy	Parcel size rule changes by adding a class	domain/parcel.ts
Factory method	One place makes reference codes and retries clashes	domain/referenceCode.ts
Observer	Emails and updates react to events, decoupled	events/eventBus.ts
State machine	Only the moves in the Task 1 state diagrams	domain/household.ts
Dependency injection	Tests swap the clock and mail sender safely	container.ts

KEY FEATURES · HOSTING

Hosted on Azure, deployed from GitHub

GITHUB

Actions pipeline

Tests every change

Builds one package

Deploys after approval

APP SERVICE · B1 PLAN

Two Node 22 web apps

Production
from main

Staging
from develop

HTTPS only, TLS 1.2 or higher

One origin: React build and API

POSTGRESQL 16 · B1MS

Two databases

`sizani_production`
`sizani_staging`

TLS required

Firewall: only the two apps

7-day automatic backups

`sizaniconnect-bb7315.azurewebsites.net`

`sizaniconnect-bb7315-staging.azurewebsites.net`

Azure for Students subscription · South Africa North region · secrets kept only in the web apps' settings

MEETING THE REQUIREMENTS

How the business rules are enforced

Rule	What it means	Enforced by
BR-01	One live registration per person, at any site	Database: unique index on ID hash
BR-02	Only active households collect a parcel	Domain rule, checked on every parcel
BR-03	A manager verifies, not whoever registered it	Manager role + four-eyes check
BR-04	One parcel per household per event	Database: unique index per event
BR-05	Parcel size follows household size	Strategy classes
BR-06	Oldest expiry issued first; expired stock never	FEFO allocator with row locks
BR-07	An event opens only once volunteers check in	Check before an event opens
BR-08	Archive after 12 months without a collection	Archive action for managers
BR-09	Every change to personal data is audited	Database: append-only audit trigger
BR-10	Check-in opens 60 minutes before a shift	Check at shift check-in

MEETING THE REQUIREMENTS

The non-functional requirements

96%

test coverage of domain, services and security code; target 70% (NFR-18)

360 px

every screen checked at phone width, no sideways scrolling (NFR-11, 12)

5.5 : 1

lowest text contrast in the app, 44 px touch targets (NFR-10)

0

ID numbers stored: only a keyed hash, and TLS everywhere (NFR-13 to 15)

1

transaction per parcel: parcel, stock and audit save together (NFR-06)

Append-only

audit log: the database refuses edits and deletes (NFR-17)

Still to measure: response times (NFR-01 to 03) and uptime (NFR-04), with a load test on staging.

MEETING THE REQUIREMENTS

Security, mapped to the Task 1 threats

Threat	Control in the code
Password guessing (T-01)	PBKDF2 at 600,000 rounds, lock after 5 failures, 10 sign-ins a minute per IP
Cross-site scripting (T-02)	Strict CSP, React encoding, lint ban on raw HTML, server validation
SQL injection (T-03)	Parameterised queries only; the lint step fails the build otherwise
Reading others' data (T-04)	Coordinators limited to their site, donors to their own records
Privilege escalation (T-05)	Deny-by-default role policy on every route
Database leak (T-06)	ID numbers kept only as a keyed hash; TLS; firewall
Request forgery (T-08)	Bearer tokens; SameSite=Strict refresh cookie; Origin check
Session theft	15-minute tokens in memory; a reused refresh token ends the session
Denial of service (T-10)	Per-IP request limit on every route; 100 KB body cap

TESTING, PIPELINES AND RELIABILITY

Automated testing on every change

115
API tests, unit and integration, on real PostgreSQL

96%
coverage; the build fails below 70%

6
front-end tests: sign-in, roles and the desk

1
parcel saved when two phones scan one slip at once

Runs on every push and pull request:

- Lint: SQL and XSS rules
- Type checks
- Tests on PostgreSQL
- 70% coverage gate
- Front-end build
- Dependency audit
- CodeQL security scan

The tests caught four real bugs before release, including a security fix that was being undone.

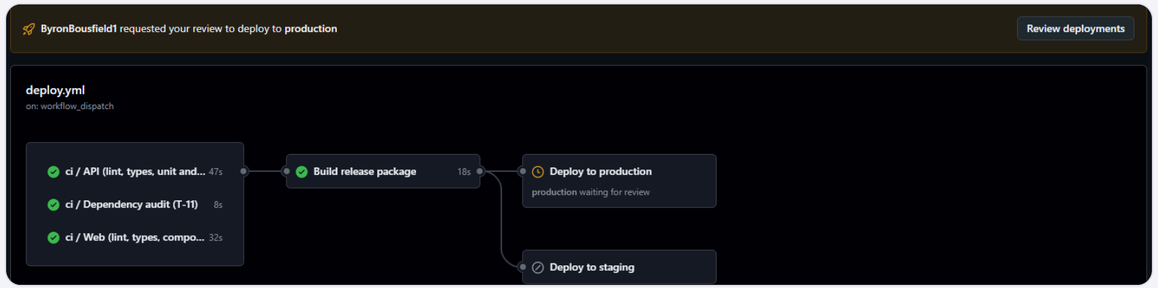
From commit to production

EVERY CHANGE

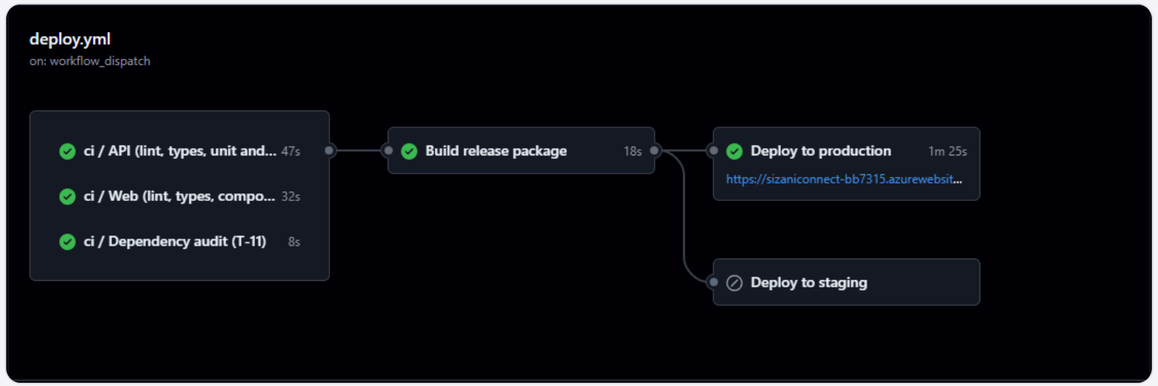
1. Branch per user story, e.g. US-09
2. Pull request into develop; CI must pass
3. Merge, build one package, deploy to staging
4. Smoke test: live, with the new commit

EVERY RELEASE

5. Merge develop into main and tag the release
6. Production waits for approval in GitHub
7. Deploy to production, same smoke test
8. Rollback: rerun Deploy on the previous tag



Step 6: production waits for approval



Release 1.0.0: checks passed, deployed to production

DECISIONS AND CHALLENGES

Technical decisions

TypeScript end to end

Node.js and React, one language throughout, replaced Task 1's ASP.NET Core. Layers, patterns, schema and API stayed as designed.

The API serves the front end

One origin: no CORS, a SameSite=Strict cookie and a 'self'-only security policy. Two origins need a cookie Safari blocks.

Race-prone rules in the database

Duplicate parcels and duplicate registrations hit unique indexes, so two phones at the same moment still cannot double-issue.

A firewall, not a private network

Azure for Students rejected the private network. Only the two apps' addresses can connect, over TLS; no code change to move later.

DECISIONS AND CHALLENGES

Challenges overcome

Challenge	What I did
Azure rejected the private database network three times	Switched to a firewall that admits only the two apps; updated the setup script and docs
CodeQL raised 3 high alerts: missing rate limiting	Added a per-IP limit to every route, with a test; the 4th alert, CSRF, is already covered
A test showed a stolen-session lock-out was being rolled back	The revocation now commits before the request is refused
A sign-in limit would lock out volunteers sharing one phone network	Token refresh exempted; its 256-bit secret cannot be guessed

DECISION REQUESTED

Ready for a pilot at Diepsloot

We ask management to approve implementation, starting with a four-week pilot at Diepsloot Extension 2 Hall, as planned in Task 1.

1
Pilot at one site, with the paper register as a backup

2
Train coordinators and two internal champions

3
Load-test staging against the response-time targets

4
Roll out to all six sites

Questions?

sizaniconnect-bb7315.azurewebsites.net
github.com/ByronBousfield1/sizaniconnect